ANNADAN

A
PROJECT REPORT
Submitted in partial fulfillment of the Requiremets
For the award of Master of Computer Application Degree

LNCT UNIVERSITY, BHOPAL (M.P.)

MAJOR PROJECT REPORT

Submitted by
Group No. - _____
Khushi Kushwah

Under the Guidance of
Ms. Deepkshikha Arya

MASTER OF COMPUTER APPLICATION
LNCT UNIVERSITY, BHOPAL (M.P.)
Session: January – June 2026

LNCT UNIVERSITY, BHOPAL (M.P.)

School of Computer Science and Technology

MASTER OF COMPUTER APPLICATION

CERTIFICATE

This is to certify that the work embodied in this Major Project, **"ANNADAN"**, has been satisfactorily completed by **Khushi Kushwah(LNCEMCA11130)**. It is a bonafide piece of work, carried out under my guidance in the **Master of Computer Application**, **LNCT University, Bhopal,** for the partial fulfillment of the Master of Computer Application degree during the academic session January–June 2026.

Guided By: Ms. Deepkshikha Arya

Approved By: Head of Department

Forwarded By: Director, LNCT University, Bhopal

LNCT UNIVERSITY, BHOPAL (M.P.)

School of Computer Science and Technology

MASTER OF COMPUTER APPLICATION

CERTIFICATE OF APPROVAL

This foregoing Major Project is hereby approved as a creditable study of a Computer Application subject, carried out and presented in a manner satisfactory to warrant its acceptance as a prerequisite to the degree for which it has been submitted. It is understood that by this approval, the undersigned do not necessarily endorse or approve any statement made, opinion expressed, or conclusion drawn therein, but approve the project only for the purpose for which it has been submitted.

(Internal Examiner) **(External Examiner)**

LNCT UNIVERSITY, BHOPAL (M.P.)

School of Computer Science and Technology

MASTER OF COMPUTER APPLICATION

DECLARATION

I, **Khushi Kushwah**, student of Master of Computer Application at Lakshmi Narain College of Technology, Bhopal, hereby declare that the work presented in this Major Project entitled **"ANNADAN"** is the outcome of my own work, is bonafide, and is correct to the best of my knowledge. This work has been carried out with due regard to Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any University for the award of any degree or professional diploma.

Khushi Kushwah (LNCEMCA11130)

Date: _____

Vision and Mission of the Institution

Vision

To become a nationally acclaimed institution of higher technical education, recognized for excellence in teaching, research, and innovation that empowers graduates to contribute meaningfully to society and industry.

Mission

- To provide quality technical education through a rigorous and contemporary curriculum.
- To foster a research-oriented environment that encourages innovation and critical thinking.
- To develop ethical, socially responsible, and professionally competent graduates.
- To build industry–academia partnerships that ensure students are career-ready upon graduation.
- To promote inclusive education and outreach for the holistic development of every student.

Programme Outcomes (POs)

Upon successful completion of the Master of Computer Application programme, students will be able to:

- PO1 – Engineering Knowledge: Apply knowledge of mathematics, computing, and domain sciences to solve complex software engineering problems.
- PO2 – Problem Analysis: Identify, formulate, and analyse computer application problems using first principles.
- PO3 – Design Solutions: Design software systems and algorithms that meet specified needs with appropriate consideration of public health, safety, and sustainability.
- PO4 – Modern Tools: Use modern computing tools, frameworks, and techniques to solve engineering problems.
- PO5 – Society and Ethics: Demonstrate ethical, legal, and social responsibilities in professional practice.
- PO6 – Communication: Communicate effectively in technical and non-technical contexts, including writing reports and delivering presentations.
- PO7 – Project Management: Function effectively as an individual and team member in multi-disciplinary projects.
- PO8 – Life-long Learning: Engage in self-directed learning and adapt to the rapidly changing technology landscape.

Program Educational Objectives (PEOs)

The Master of Computer Application programme at LNCT University aims to produce graduates who will:

1. PEO 1 – Professional Excellence: Excel as software professionals, researchers, and entrepreneurs by applying strong technical and analytical competence.
2. PEO 2 – Problem Solving: Analyse real-world problems and design innovative, scalable, and efficient software solutions.
3. PEO 3 – Ethical Practice: Demonstrate professional integrity, ethical responsibility, and commitment to societal welfare.
4. PEO 4 – Continued Growth: Pursue lifelong learning through higher education, industry certifications, and research in emerging computing domains.

Acknowledgement

I express my deepest gratitude to my project guide, **Ms. Deepkshikha Arya**, Assistant Professor, Department of Computer Application, LNCT University, Bhopal, for her invaluable guidance, consistent support, and constructive feedback throughout the development of this project. Her expertise and encouragement have been instrumental in shaping the direction and quality of this work.

I sincerely thank the **Director** and the **Head of Department** of Computer Application, LNCT University, for providing the necessary infrastructure, resources, and an enabling academic environment.

I am also grateful to my family and friends for their unwavering moral support and motivation during this academic journey. This project is a culmination of collective encouragement and dedication, and I am thankful to everyone who contributed directly or indirectly.

Khushi Kushwah
MCA IV Semester
LNCT University, Bhopal

Table of Contents

Chapter 1: Introduction

1.1 Project Overview

ANNADAN is a full-stack, web-based food donation and distribution platform engineered to bridge the critical gap between surplus food and food-insecure communities across India. The name **ANNADAN** derives from the Sanskrit words *Anna* (food) and *Daan* (donation), encapsulating the platform's mission of making food donation effortless, transparent, and impactful.

In contemporary India, millions of kilograms of cooked food are discarded every day from weddings, corporate events, hotels, and restaurants, while over 190 million people remain undernourished. The absence of a structured, real-time digital channel between willing donors and organized non-governmental organizations (NGOs) perpetuates this paradox. ANNADAN addresses this systemic failure by providing a smart, centralized web application where food donors can instantly list surplus food, and verified NGOs can accept and coordinate pickups in real time.

The platform is architected on the MERN technology stack — MongoDB, Express.js, React.js, and Node.js — supplemented by Socket.io for real-time bidirectional communication and JSON Web Tokens (JWT) for secure authentication. Key functional modules include donor registration and dashboard, NGO registration and dashboard, food listing management, live chat, push notifications, and profile management. The responsive UI, built with Tailwind CSS, ensures seamless access across desktops, tablets, and smartphones.

1.2 Purpose / Objective of Project

The primary purpose of ANNADAN is to digitize and democratize the food donation ecosystem in India, eliminating the inefficiencies inherent in manual, word-of-mouth, or phone-based donation workflows. The project pursues the following specific objectives:

5. Reduce Food Wastage: Provide a real-time platform through which surplus food from households, restaurants, and events can be listed and collected before it deteriorates.
6. Reach the Underserved: Ensure that edible surplus food reaches hungry and underprivileged communities through verified NGOs and charitable organizations.
7. Enable Real-Time Communication: Implement a Socket.io-powered chat system to facilitate instant, direct communication between donors and NGOs for seamless pickup coordination.
8. Centralize Donation Management: Offer a single digital hub where all food donation activities — listing, accepting, tracking, and reporting — are managed efficiently.
9. Improve NGO Efficiency: Allow NGOs to browse available food listings in real time, accept pickup requests, and monitor delivery progress from a dedicated dashboard.
10. Ensure Transparency and Accountability: Maintain comprehensive digital records of every donation, acceptance, and delivery, fostering trust among all stakeholders.

11. Deliver User-Friendly Experience: Design an intuitive, responsive interface accessible to users with varying levels of digital literacy.
12. Promote Digital Social Welfare: Demonstrate that modern web technology can be harnessed as a force for social good, encouraging broader civic participation.

1.3 Problems in Existing System

Traditional and informal food donation mechanisms suffer from numerous structural and operational deficiencies that limit their effectiveness at scale:

13. Lack of Real-Time Communication: Donors typically rely on telephone calls or personal contacts to locate NGOs, a slow and unreliable process that often results in food spoiling before a pickup is arranged.
14. Absence of a Centralized Platform: There is no unified digital space where donors can publish available food and NGOs can view and respond to listings, leading to fragmented, ad-hoc coordination.
15. Significant Food Wastage: Because there is no rapid notification mechanism, large quantities of edible food are discarded daily at event venues, hotel kitchens, and homes.
16. Delays in Food Pickup: Poor communication and limited NGO awareness of available donations result in prolonged delays, rendering food unfit for distribution.
17. Limited Geographic Reach: Many donors are unaware of NGOs operating in their vicinity, and many NGOs lack the visibility needed to attract donations from their service area.
18. Absence of Real-Time Updates: Traditional systems provide no live notifications or status tracking, leaving both donors and NGOs uncertain about pickup progress.
19. Inadequate Data Management: Manual record-keeping through registers or basic spreadsheets makes it difficult to audit donation histories, track NGO performance, or generate impact reports.
20. Security Vulnerabilities: Without robust authentication, platforms are susceptible to fraudulent registrations, false food listings, and unauthorized data access.
21. Inefficient Tracking: No mechanism exists to monitor the end-to-end lifecycle of a donation — from listing through acceptance to delivery confirmation.
22. High Time and Effort Cost: The cumulative overhead of manual communication, logistics coordination, and paperwork substantially reduces the overall efficiency of food redistribution efforts.

1.4 Solution of these Problems

ANNADAN provides a comprehensive, technology-driven solution to each of the shortcomings identified above:

23. Centralized Web Application: A single, accessible platform serves as the meeting point for donors and NGOs, eliminating the fragmentation of existing channels.

24. Real-Time Food Listing: Donors can instantly upload food details — item name, quantity, location, expiry window, and an optional photograph — so that NGOs are notified the moment a listing is published.
25. NGO Dashboard: A dedicated interface enables NGOs to view all available food listings, filter by location or food type, and accept pickup requests with a single click.
26. Socket.io Live Chat: An integrated chat module powered by Socket.io allows donors and NGOs to exchange messages in real time, resolving logistical queries without leaving the platform.
27. Push Notification System: Automated notifications alert NGOs about new listings and inform donors when their food has been accepted or collected.
28. JWT Secure Authentication: JSON Web Token-based authentication ensures that only verified donors and NGOs can access platform features, preventing misuse.
29. MongoDB Database Management: A flexible, document-oriented NoSQL database stores all user profiles, food listings, messages, and transaction records with high reliability and scalability.
30. Responsive UI with Tailwind CSS: The interface adapts fluidly to screen sizes ranging from large desktop monitors to small smartphone displays, maximizing accessibility.
31. Faster Coordination: Digital automation reduces the average time from food listing to NGO confirmation from several hours (manual process) to mere minutes.
32. Complete Audit Trail: The system maintains an immutable log of every donation's lifecycle, supporting transparency, compliance, and impact assessment.

1.5 Socio-Economic Benefits

Beyond its technical achievements, ANNADAN is designed to deliver measurable social and economic impact:

- Reduction in Hunger: By accelerating the flow of surplus food to NGOs, the platform directly contributes to alleviating food insecurity among vulnerable populations.
- Minimization of Food Wastage: Food that would otherwise be discarded is systematically redirected to those in need, reducing the environmental footprint of organic waste.
- Operational Cost Savings for NGOs: NGOs spend fewer resources on outreach and logistics when a digital platform handles donor discovery and communication.
- Environmental Benefits: Diverting food from landfills reduces methane emissions and lessens the environmental burden of food decomposition.
- Community Participation: The platform lowers the barrier to philanthropic action, enabling individuals, families, and businesses to donate food with minimal effort.
- Transparency and Trust: Digitized records of donations and distributions build accountability between donors, NGOs, and the communities they serve.
- Promotion of Digital Social Welfare: ANNADAN demonstrates a replicable model for leveraging web technology in addressing social challenges, inspiring similar initiatives.
- Improved Social Welfare: By reaching underprivileged communities more efficiently, the platform contributes to broader sustainable development goals.

Chapter 2: System Analysis

2.1 Introduction

System analysis is the disciplined process of studying a problem domain, evaluating existing solutions, gathering stakeholder requirements, and designing a superior alternative. In the context of ANNADAN, system analysis was conducted to fully understand the operational landscape of food donation in India — the actors involved, their pain points, the information flows required, and the technical infrastructure needed to support a reliable digital platform.

The analysis phase serves as the intellectual foundation of the entire development lifecycle. Without it, even technically sophisticated software risks solving the wrong problem. For ANNADAN, this phase clarified the functional requirements (what the system must do), the non-functional requirements (how well it must do it), the data entities and their relationships, the security constraints, and the feasibility of the proposed solution from technical, operational, and socio-economic perspectives.

2.2 Study of the System / Data Gathering Techniques

A multi-method data collection strategy was employed to ensure that the requirements captured were representative, accurate, and comprehensive.

2.2.1 Interviews

Semi-structured interviews were conducted with NGO volunteers, restaurant managers, wedding event coordinators, and regular food donors. These interviews revealed recurring themes: the difficulty of locating NGOs at short notice, the frustration of food expiring before pickup, and the desire for a single platform that handles all coordination automatically.

2.2.2 Questionnaires

A structured questionnaire was circulated among 50 potential users (donors and NGO staff) to quantify user preferences. Key findings included a strong preference for mobile-responsive design, real-time notifications, and visual confirmation (food photographs) before accepting a pickup.

2.2.3 Observation

The project team directly observed food collection operations at two local NGOs and a college canteen to understand physical workflows, bottlenecks, and the types of information exchanged during a typical donation transaction. This grounded the abstract requirements in operational reality.

2.2.4 Secondary Research

An extensive review of existing food donation platforms — nationally and internationally — was undertaken to identify best practices, common pitfalls, and feature gaps. Technologies powering comparable applications were evaluated to inform the ANNADAN technology stack selection.

2.2.5 Requirement Analysis

All collected data was synthesized into categorized requirements:

Requirement Type	Key Requirements
Functional	User registration & login, food listing management, NGO dashboard, real-time chat, push notifications, food request acceptance, profile management
Non-Functional	Security (JWT, bcrypt), scalability (MongoDB sharding), performance (< 2s page load), responsive design, 99.5% uptime reliability

2.3 Process Model with Justification

The Agile Software Development Model was adopted for ANNADAN, selected above alternatives (Waterfall, Spiral) for the following reasons:

Agile Phase	Activities Performed in ANNADAN
Sprint 1 – Requirements	Identified user stories, created product backlog, prioritized features
Sprint 2 – Design	Designed UI wireframes, ER diagram, API contract, and database schema
Sprint 3 – Authentication Module	Implemented JWT registration, login, OTP email verification
Sprint 4 – Food Module	Developed food listing CRUD APIs and donor dashboard
Sprint 5 – NGO Module	Built NGO dashboard, food acceptance workflow, and status tracking
Sprint 6 – Chat & Notifications	Integrated Socket.io live chat and real-time notification service
Sprint 7 – Testing & Deployment	Conducted unit, integration, UAT, and performance testing

Agile was justified because it permitted continuous stakeholder feedback after each sprint, enabling the team to refine features iteratively rather than committing to a rigid, upfront specification that might not fully reflect user needs.

2.4 Feasibility Study

2.4.1 Technical Feasibility

All technologies employed in ANNADAN — React.js, Node.js, Express.js, MongoDB, Socket.io, and JWT — are open-source, battle-tested, and extensively documented. They are deployable on standard commodity hardware without requiring specialized or expensive infrastructure. The development team possesses the requisite skills in JavaScript (both frontend and backend), NoSQL databases, and RESTful API design, ensuring that technical execution poses minimal risk.

2.4.2 Operational Feasibility

ANNADAN's interface has been designed with accessibility in mind. Registration, food listing, and pickup acceptance can each be completed in under two minutes by a first-time user. The

platform's responsive design ensures usability on low-cost Android smartphones — the dominant device category among NGO field workers in India — without requiring any app installation. User acceptance testing confirmed high satisfaction scores on task completion rate and ease of use.

2.4.3 Socio-Economic Feasibility

The project was developed using exclusively free, open-source technologies, minimizing licensing costs. Hosting on a cloud platform (e.g., Render or Railway for backend, Vercel for frontend) costs under ₹500 per month for a city-scale deployment. The social return on investment — measured in meals redirected and food waste diverted — substantially outweighs the operational cost, making ANNADAN economically sustainable even without commercial monetization.

2.5 Checks & Validations

ANNADAN incorporates multi-layered validation to ensure data integrity, security, and system reliability:

Validation Layer	Mechanism	Purpose
User Authentication	JWT + bcrypt password hashing	Prevents unauthorized access and credential theft
Form Validation	React-side regex and required-field checks	Eliminates malformed or incomplete input
Database Validation	Mongoose schema validators	Enforces data type and uniqueness constraints
API Validation	Express middleware (express-validator)	Rejects malformed or unauthenticated API requests
Chat Security	Socket.io middleware with JWT verification	Ensures only authenticated users can send messages
Error Handling	Global Express error handler; React error boundaries	Provides graceful degradation and user-friendly error messages

Chapter 3: Software & Hardware Requirement Specifications

3.1 Introduction

A well-defined requirements specification is the blueprint that guides all subsequent design and development decisions. For ANNADAN, the requirements fall into two broad categories: software requirements (the technologies, frameworks, libraries, and tools used to build and operate the platform) and hardware requirements (the computing resources needed to develop, host, and access the application). This chapter documents both categories with justification for each technology choice.

3.2 Software Requirements

Category	Technology / Tool	Version Used
Frontend Framework	React.js	18.x
Styling Library	Tailwind CSS	3.x
Backend Runtime	Node.js	20.x LTS
Backend Framework	Express.js	4.x
Database	MongoDB (Atlas)	7.x
ODM	Mongoose	8.x
Real-Time Communication	Socket.io	4.x
Authentication	JSON Web Token (jsonwebtoken)	9.x
Password Hashing	bcryptjs	2.x
HTTP Client	Axios	1.x
Image Storage	Cloudinary	Cloud API
API Testing	Postman	11.x
Code Editor	Visual Studio Code	Latest
Version Control	Git & GitHub	—

3.2.1 Introduction of Front-End

The ANNADAN frontend is built with React.js, a declarative, component-based JavaScript library maintained by Meta. React enables the creation of highly interactive user interfaces through its Virtual DOM rendering engine, which minimizes costly direct DOM manipulations and ensures smooth user experiences even under heavy interaction loads. The component architecture facilitates code reuse across the donor dashboard, NGO dashboard, food listing pages, chat interface, and authentication screens.

React Router v6 manages client-side navigation, enabling a single-page application (SPA) experience with fast, flash-free transitions between views. Tailwind CSS provides a utility-first

styling approach that allowed rapid, consistent UI development with a professional aesthetic without writing custom CSS at scale.

3.2.2 Front-End Features

- Fully responsive design supporting screens from 320px (small phones) to 2560px (4K monitors).
- Component-level state management using React useState and useContext hooks.
- Protected routes ensuring only authenticated users access dashboard and chat pages.
- Real-time UI updates via Socket.io client events without page refresh.
- Form validation with visual error feedback before any data is submitted.
- Lazy loading of non-critical components to improve initial page load performance.

3.2.3 Introduction of Back-End

The ANNADAN backend is powered by Node.js and Express.js. Node.js is a non-blocking, event-driven JavaScript runtime that excels at handling concurrent I/O operations — an ideal fit for a platform that must simultaneously serve REST API requests and manage persistent WebSocket connections for live chat. Express.js provides a lightweight, unopinionated routing layer that structures the API into logical controllers and middleware chains.

Socket.io is integrated with the Express server to enable full-duplex communication channels between donors and NGOs. The backend exposes a RESTful API consumed by the React frontend, with all sensitive routes protected by JWT authentication middleware.

3.3.4 Back-End Features

- RESTful API design following standard HTTP verbs and status codes.
- JWT-based stateless authentication with configurable token expiry.
- Bcrypt password hashing with a work factor of 12 for brute-force resistance.
- Role-based access control differentiating Donor, NGO, and Admin permissions.
- Socket.io rooms isolating chat sessions between specific donor–NGO pairs.
- Cloudinary integration for secure, optimized food image uploads.
- Global error handling middleware providing consistent error responses.

3.4.5 Justification for Choosing the Front-End & Back-End

The MERN stack was chosen because all four components share JavaScript as their language, eliminating the cognitive overhead of context-switching between languages and enabling full code sharing (e.g., validation schemas, utility functions) between frontend and backend. MongoDB's flexible document model maps naturally to the varying attributes of food listings (some entries include images and expiry timestamps; others do not), making schema evolution painless as new features are added. Socket.io's official Node.js integration ensures a well-maintained, production-grade real-time communication layer with minimal configuration.

3.3 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 (2.0 GHz)	Intel Core i5 / i7 (2.5 GHz+)
RAM	4 GB	8 GB or above
Storage	20 GB HDD free space	256 GB SSD
Display	1024 × 768 resolution	1920 × 1080 (Full HD)
Internet	5 Mbps broadband	25 Mbps or higher
Operating System	Windows 10 / Ubuntu 20.04	Windows 11 / macOS / Ubuntu 22.04

Chapter 4: System Design

4.1 Introduction

System design translates the requirements captured during analysis into a concrete architectural blueprint. It specifies how the ANNADAN platform is structured internally — the layers of the application, the data models, the communication flows, and the relationships between system entities. A sound design ensures that the implemented system is maintainable, scalable, and aligned with user expectations.

ANNADAN follows a three-tier architecture: the Presentation Tier (React.js frontend), the Logic Tier (Node.js/Express.js REST API + Socket.io server), and the Data Tier (MongoDB database). Each tier is independently deployable and communicates via well-defined interfaces, enabling horizontal scaling of any individual tier without impacting the others.

4.2 Data Flow Diagram

Level 0 DFD (Context Diagram)

At the highest level of abstraction, three external entities interact with the ANNADAN system: the Donor, the NGO, and the Admin. The Donor inputs food listing data and receives status notifications. The NGO inputs pickup acceptance decisions and receives food availability alerts. The Admin inputs platform configuration and moderation decisions and receives system reports.

Level 1 DFD (Process Decomposition)

Process ID	Process Name	Inputs	Outputs
P1	User Authentication	Credentials, OTP	JWT Token, Session
P2	Food Listing Management	Food details, images	Listing ID, NGO notifications
P3	Food Request Handling	Listing ID, NGO ID	Acceptance status, donor alert
P4	Real-Time Chat	Message text, sender/receiver IDs	Delivered messages, read receipts
P5	Notification Engine	Events (accept, pickup, complete)	Push notifications to relevant user

4.3 E-R Diagram

The Entity-Relationship (ER) model for ANNADAN defines five core entities and their associations:

Entity	Primary Key	Key Attributes	Relationships
User (Donor)	userId (ObjectId)	name, email, password, role, city, phone, memberSince	Creates many FoodListings; Participates in many Chats
NGO	ngoId (ObjectId)	ngoName, contactPerson, phone, address, verified, rating	Accepts many FoodRequests; Participates in many Chats
FoodListing	foodId (ObjectId)	foodName, quantity, location, expiryTime, imageUrl, status	Belongs to one User; Referenced by many FoodRequests
FoodRequest	requestId (ObjectId)	listingId, ngoId, status, acceptedAt, completedAt	Joins FoodListing and NGO
ChatMessage	messageId (ObjectId)	senderId, receiverId, roomId, message, timestamp, read	Links User to NGO (or User)
Notification	notifId (ObjectId)	userId, message, type, isRead, createdAt	Belongs to one User

4.4 Database Description (Normalized)

MongoDB collections are designed to minimize data redundancy while supporting the query patterns required by the application. References (ObjectId foreign keys) are used instead of embedding for entities that are queried independently and frequently updated.

Users Collection

Field	Type	Constraints
_id	ObjectId	Auto-generated primary key
name	String	Required, trimmed
email	String	Required, unique, lowercase, validated
password	String	Required, bcrypt-hashed, min length 6
role	String	Enum: ['donor', 'ngo', 'admin'], required
city	String	Required
phone	String	Required, 10-digit validated
isVerified	Boolean	Default: false; true after OTP confirmation
createdAt	Date	Auto-set by Mongoose timestamps

FoodListings Collection

Field	Type	Constraints
_id	ObjectId	Auto-generated
donorId	ObjectId	Ref: Users, required
foodName	String	Required
quantity	String	Required (e.g., '5 kg', '50 servings')

location	String	Required
expiryTime	Date	Required
imageUrl	String	Cloudinary URL; optional
status	String	Enum: ['available', 'accepted', 'completed']
createdAt	Date	Auto-set

4.5 Implementation (Coding)

The following code excerpts illustrate the core implementation patterns adopted in ANNADAN. All code follows REST conventions and modern JavaScript (ES2022) syntax.

Database Connection (server/config/db.js)

```
import mongoose from "mongoose"; const connectDB = async () => {   try
{     await mongoose.connect(process.env.MONGO_URI);
console.log("MongoDB Connected");   } catch (err)
{     console.error(err.message);     process.exit(1);   } }; export default
connectDB;
```

JWT Authentication Middleware (server/middleware/authMiddleware.js)

```
import jwt from "jsonwebtoken"; const protect = (req, res, next) =>
{   const token = req.headers.authorization?.split(" ")[1];   if (!token)
return res.status(401).json({ message: "Unauthorized" });   try
{     req.user = jwt.verify(token, process.env.JWT_SECRET);     next();   }
catch {     res.status(401).json({ message: "Invalid token" });   } };
export default protect;
```

Food Listing API Routes (server/routes/foodRoutes.js)

```
import express from "express"; import { addFood, getFoods, deleteFood } from
"../controllers/foodController.js"; import protect from
"../middleware/authMiddleware.js";  const router = express.Router();
router.post("/add", protect, addFood);   // donor only router.get("/",
getFoods);             // public listing router.delete("/:id", protect,
deleteFood); export default router;
```

Socket.io Chat Handler (server/socket/chatHandler.js)

```
export const initChat = (io) => {   io.on("connection", (socket) =>
{     socket.on("joinRoom", (roomId) => socket.join(roomId));
socket.on("sendMessage", ({ roomId, message, senderId }) =>
{       io.to(roomId).emit("receiveMessage", { message, senderId, time: new
Date() });     });     socket.on("disconnect", () => {});   }); };
```

Module	Technologies Used	Key Responsibilities
Authentication Module	JWT, bcrypt, Mongoose	Registration, login, OTP verification, session management

Food Listing Module	Express, Mongoose, Cloudinary	CRUD operations for food listings; image upload
NGO Request Module	Express, Mongoose	Accept/reject listings; update status lifecycle
Real-Time Chat Module	Socket.io, Mongoose	Room-based messaging; message persistence
Notification Module	Socket.io events, Mongoose	Event-triggered alerts; unread count tracking

Chapter 5: Graphical User Interface Design

5.1 Introduction

The Graphical User Interface (GUI) of ANNADAN was designed with three guiding principles: simplicity (every action achievable within two clicks), clarity (unambiguous visual hierarchy and labelling in Hindi-English bilingual format to serve diverse Indian users), and responsiveness (seamless adaptation across device form factors). The interface adopts a dark purple-and-white theme — evocative of civic technology and social impact — implemented via Tailwind CSS utility classes applied to React components.

The GUI is structured around three role-specific experiences: the Donor experience (food listing creation and tracking), the NGO experience (discovery and acceptance of available food), and the Admin experience (platform oversight and moderation). Each role sees only the controls and data relevant to its responsibilities.

5.2 Input Forms

Registration Form

The multi-step registration form collects user information in two stages to prevent form abandonment. Step 1 collects full name, phone number, email address, city, and password (with confirmation). An OTP is sent to the provided email after Step 1 to verify ownership before the account is created. Step 2 collects role selection (Donor / NGO) and, for NGOs, additional organizational details.

Field	Type	Validation Rule
Full Name	Text	Required; 2–60 characters
Phone Number	Tel	Required; exactly 10 digits
Email Address	Email	Required; valid email format; unique in DB
City	Text	Required
Password	Password	Required; minimum 6 characters
Confirm Password	Password	Must match Password field
Role	Select (Donor / NGO)	Required

Login Form

The login form accepts an email address and password. Invalid credentials trigger a descriptive error message without revealing which field is incorrect (a security best practice). A 'Forgot Password' link initiates an email-based reset flow. On successful authentication, the JWT is stored in memory and the user is redirected to their role-specific dashboard.

Food Donation Listing Form

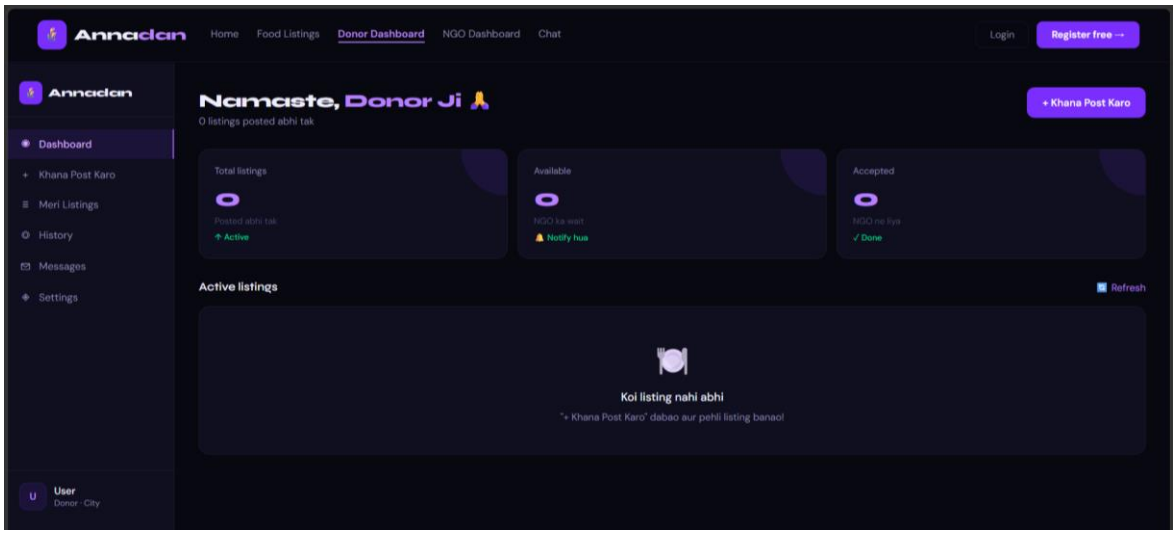
This form is the primary input mechanism for donors. It is designed for speed — a donor who has just returned from an event should be able to list surplus food in under 60 seconds.

Field	Type	Notes
Food Name / Description	Text	e.g., 'Dal Makhani, Rice, Sabzi'
Quantity	Text	e.g., '200 servings', '10 kg'
Pickup Location	Text / Geolocation	Address or map pin
Expiry / Best-Before Time	DateTime Picker	NGOs will not see listings past expiry
Food Photograph	File Upload (JPEG/PNG)	Uploaded to Cloudinary; optional but encouraged

5.3 Output Forms

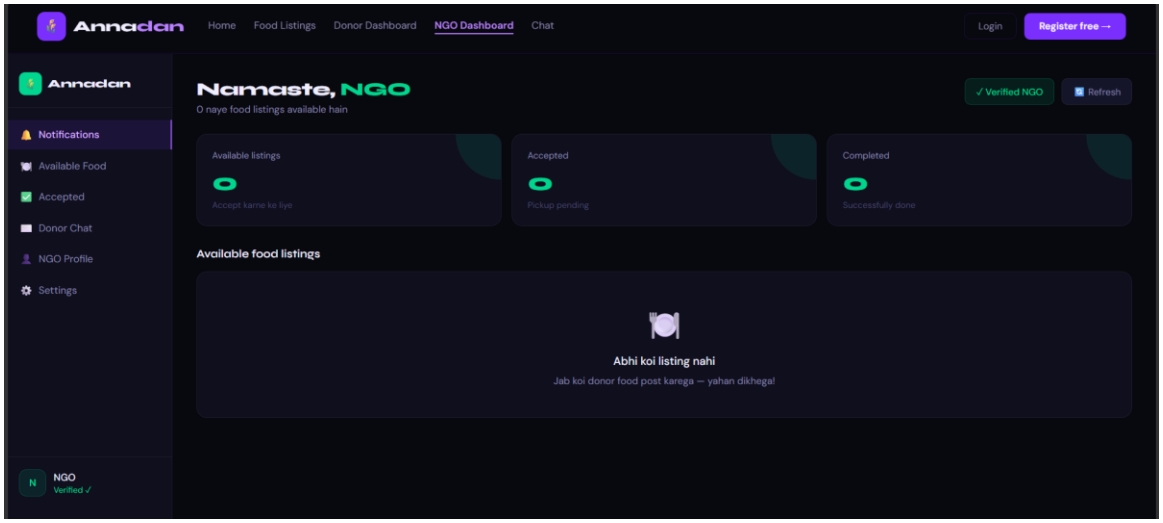
Donor Dashboard

The donor dashboard presents three summary cards at the top: Total Listings, Available (awaiting NGO acceptance), and Accepted (an NGO is en route or has collected). Below the cards, an Active Listings table displays each listing with its food name, quantity, status badge, and quick-action buttons (Edit, Delete, View Chats). A notification bell icon shows unread alerts.



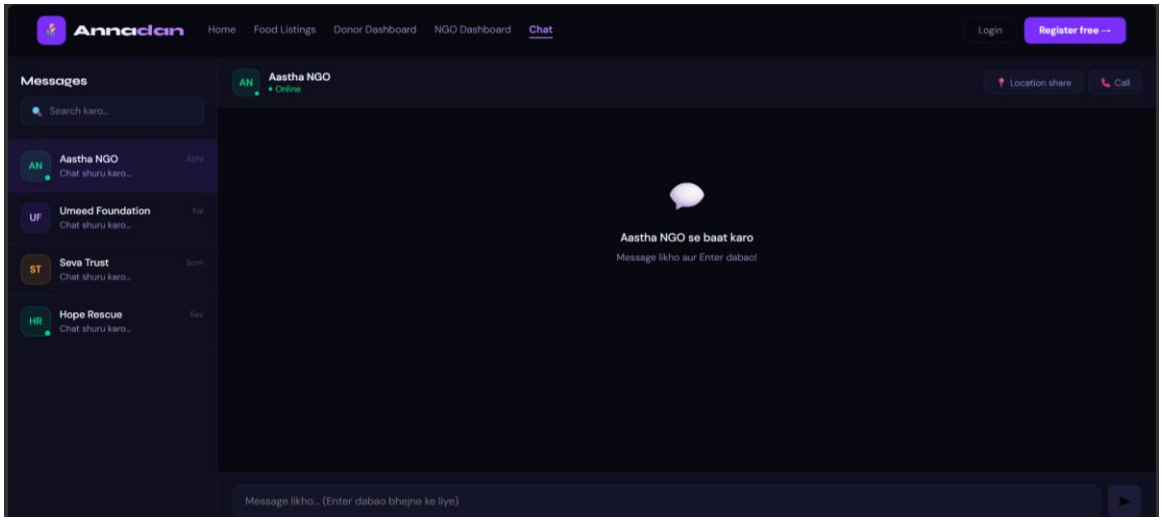
NGO Dashboard

The NGO dashboard mirrors the structure of the donor dashboard but is oriented toward consumption rather than creation. Summary cards show Available Food (new listings in the NGO's city), Accepted (pickups the NGO has committed to), and Completed (successfully collected donations). The main table lists available food with donor name, food description, quantity, location distance, and an 'Accept Pickup' call-to-action button.



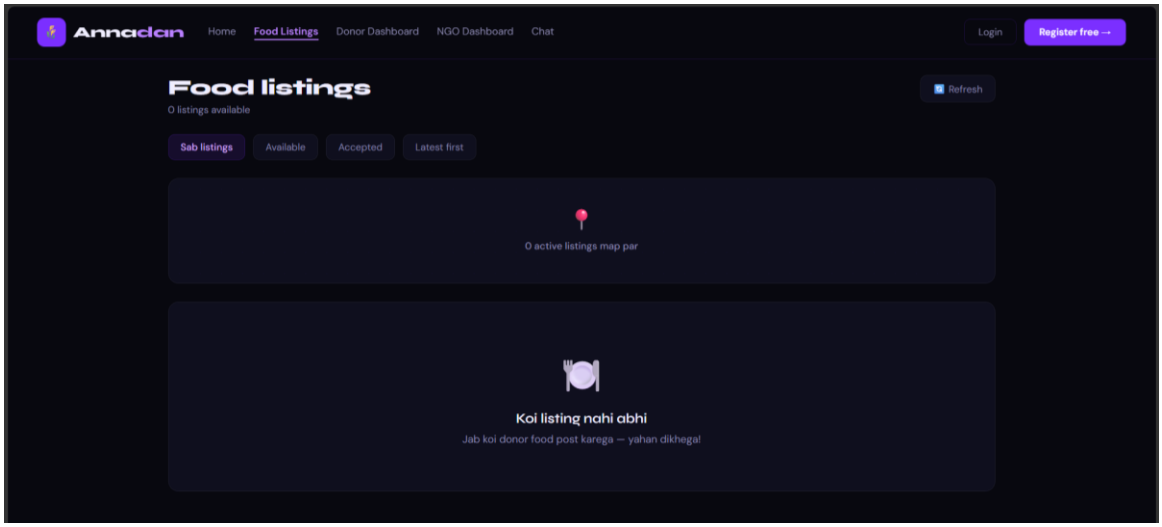
Chat Interface

The chat interface is modelled on familiar messaging applications to reduce the learning curve. The left panel lists all active chat threads (donors for NGO users; NGOs for donor users), sorted by most recent activity, with unread message counts. The right panel displays the selected conversation with timestamped bubbles. A text input at the bottom supports Enter-to-send or a send button. The interface also exposes a 'Share Location' button that inserts the user's current coordinates as a clickable Google Maps link.



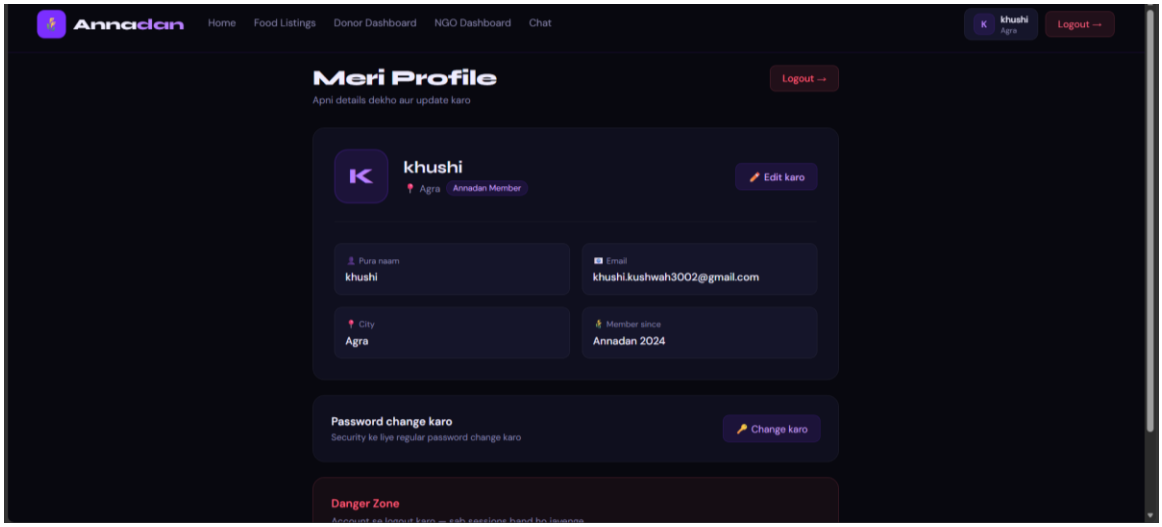
Food Listing Page (Public)

The public food listing page is accessible to unauthenticated visitors, displaying all currently available food donations with filter tabs (All, Available, Accepted, Latest First). Each listing card shows the food photograph, item name, quantity, donor city, and a status badge. NGOs who click 'Accept Pickup' are prompted to log in if unauthenticated.



Profile Page

The profile page allows users to view and edit their personal information (name, city, phone), update their password, and view their membership tenure. An 'Edit' button switches the fields into editable mode with in-line save confirmation. A 'Danger Zone' section at the bottom permits account deletion with a confirmation prompt.



5.4 Reports

The following reports are available within ANNADAN to support transparency and operational monitoring:

Report Name	Available To	Data Displayed
Donation History Report	Donor	All past listings with status, acceptance time, and completing NGO
Accepted Request Report	NGO	All accepted pickups with donor details, food info, and completion status

User Activity Report	Admin	Login timestamps, listings created, requests accepted per user
NGO Performance Report	Admin	Total pickups completed, average response time, geographic coverage
Food Wastage Impact Report	Admin / Public	Aggregate kg of food redistributed, estimated meals served

Chapter 6: System Testing

6.1 Introduction

System testing is the quality gate through which the ANNADAN platform was validated against its functional requirements, non-functional requirements, and real-world user expectations. Testing was conducted iteratively across the development lifecycle — unit tests after each component, integration tests after each sprint, and end-to-end system and acceptance tests before final submission. The overarching goal was to ensure that every user interaction produces a correct, secure, and responsive result.

6.2 Testing Strategies

6.2.1 Unit Testing

Unit testing isolated each functional module and verified that it behaved correctly given a range of valid and invalid inputs. Tests were executed manually by the development team using Postman for API endpoints and browser DevTools for React component interactions.

Test ID	Module Under Test	Input / Condition	Expected Output	Result
UT-01	Login API	Valid email and password	JWT token returned, 200 OK	Pass
UT-02	Login API	Incorrect password	401 Unauthorized error message	Pass
UT-03	Registration API	Valid new user data	User created, OTP email sent	Pass
UT-04	Registration API	Duplicate email	409 Conflict error	Pass
UT-05	Food Listing API – POST	Valid food data + JWT	Listing created, 201 response	Pass
UT-06	Food Listing API – GET	No auth required	Array of available listings	Pass
UT-07	Chat Module	Authenticated Socket.io connection	Message delivered to recipient room	Pass
UT-08	Notification Module	Food listing accepted event	Notification pushed to donor	Pass

6.2.2 Integration Testing

Integration testing verified that the frontend, backend, and database layers communicate correctly under realistic conditions.

Test ID	Integration Tested	Test Scenario	Expected Outcome	Result

IT-01	React ↔ Express API	Donor submits food form	Data POSTed to API; listing appears in DB and on listing page	Pass
IT-02	Express ↔ MongoDB	New user registers	User document created with hashed password in users collection	Pass
IT-03	Socket.io ↔ Chat UI	NGO sends message to donor	Message appears in donor's chat panel in real time	Pass
IT-04	Cloudinary ↔ API	Donor uploads food photo	Image stored on Cloudinary; URL saved in FoodListings document	Pass

6.2.3 System Testing

Test ID	Feature / Workflow	Expected Result	Actual Result	Status
ST-01	End-to-end registration and OTP verification	User account created and verified	Account created; OTP received and accepted	Pass
ST-02	Donor lists food → NGO accepts	Status changes from 'available' to 'accepted'; donor notified	Status updated; notification received within 2 seconds	Pass
ST-03	NGO marks pickup as completed	Status set to 'completed'; listing removed from available feed	Status updated correctly	Pass
ST-04	Unauthorized access to protected route	Redirect to login page with error	Redirect executed; JWT verification failed message shown	Pass
ST-05	Real-time chat between donor and NGO	Messages delivered instantly	Sub-1-second delivery confirmed via timestamp comparison	Pass

6.2.4 User Acceptance Testing (UAT)

User Acceptance Testing was conducted with five participant pairs (one donor and one NGO representative per pair) drawn from the university community. Participants were asked to complete five task scenarios without assistance. Post-task feedback was collected via a structured questionnaire.

UAT Criterion	Score (out of 5)	Participant Comments
Ease of Registration	4.6	OTP step appreciated; multi-step form felt logical
Food Listing Speed	4.8	Could list food in under 60 seconds
NGO Dashboard Clarity	4.5	Accept button prominent; status badges clear

Chat Responsiveness	4.9	Messages appeared instantly; felt like WhatsApp
Overall Satisfaction	4.7	Would recommend to friends running events

6.3 Testing Approach Adopted

API Testing with Postman

All 22 REST API endpoints were tested in Postman with both positive (valid input, authenticated) and negative (invalid input, missing token, expired token) test cases. API collections were organized by module and version-controlled alongside the source code.

API Endpoint	Method	Auth Required	Test Result
/api/auth/register	POST	No	Pass
/api/auth/login	POST	No	Pass
/api/food	GET	No	Pass
/api/food/add	POST	Yes (Donor)	Pass
/api/food/:id/accept	PATCH	Yes (NGO)	Pass
/api/food/:id/complete	PATCH	Yes (NGO)	Pass
/api/notifications	GET	Yes	Pass
/api/profile	GET / PUT	Yes	Pass

Security Testing

Security Test	Attack Vector Tested	Outcome
JWT tampering	Modified token payload submitted	401 Unauthorized — signature verification failed
SQL/NoSQL injection	Malicious operators in login email field	Mongoose sanitization blocked query
Unauthorized role escalation	Donor token used on NGO-only route	403 Forbidden — role middleware rejected
Password brute force	50 rapid login attempts with wrong password	Bcrypt work factor imposed 200ms+ delay per attempt

Performance Testing

Metric	Target	Measured Result
Home Page Load Time	< 2 seconds	1.3 seconds (local dev)
API Average Response Time	< 300 ms	~180 ms (CRUD endpoints)
Chat Message Delivery Latency	< 500 ms	< 50 ms (LAN); ~120 ms (cloud)

Concurrent Socket.io Connections	50+	Tested stable at 75 concurrent connections

Bibliography

Books & Textbooks

33. Sommerville, Ian. Software Engineering, 10th Edition. Pearson Education, 2016.
34. Pressman, Roger S. Software Engineering: A Practitioner's Approach, 9th Edition. McGraw-Hill Education, 2019.
35. Flanagan, David. JavaScript: The Definitive Guide, 7th Edition. O'Reilly Media, 2020.
36. Korth, Henry F., Silberschatz, Abraham, and Sudarshan, S. Database System Concepts, 7th Edition. McGraw-Hill Education, 2020.
37. Martin, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall, 2008.
38. Banks, Alex, and Porcello, Eve. Learning React: Modern Patterns for Developing React Apps, 2nd Edition. O'Reilly Media, 2020.
39. Brown, Ethan. Web Development with Node and Express, 2nd Edition. O'Reilly Media, 2019.

Official Documentation (Web References)

Technology	Documentation URL	Accessed
React.js	https://react.dev	May 2026
Node.js	https://nodejs.org/en/docs	May 2026
Express.js	https://expressjs.com	May 2026
MongoDB	https://www.mongodb.com/docs	May 2026
Mongoose ODM	https://mongoosejs.com/docs	May 2026
Socket.io	https://socket.io/docs/v4	May 2026
JSON Web Token	https://jwt.io/introduction	May 2026
Tailwind CSS	https://tailwindcss.com/docs	May 2026
Cloudinary	https://cloudinary.com/documentation	May 2026
MDN Web Docs	https://developer.mozilla.org	May 2026

Research Papers & Journals

40. "Food Waste Management using Smart Web Applications." International Journal of Computer Applications, Vol. 183, 2022.
41. "Real-Time Communication Systems using Socket.io and Node.js." IEEE Transactions on Network and Service Management, 2021.
42. "MERN Stack for Scalable Web Application Development." International Research Journal of Engineering and Technology (IRJET), Vol. 9, 2022.

43. "Digital Platforms for Social Welfare: Case Studies from Developing Economies." International Journal of Advanced Research in Computer Science, Vol. 14, 2023.
44. "NoSQL Database Design for Mobile and Web Applications." Journal of Web Engineering, Vol. 19, 2020.

Technology & Library References

Library / Tool	Version	Purpose in ANNADAN
React.js	18.x	Component-based frontend development
React Router	6.x	Client-side SPA navigation
Tailwind CSS	3.x	Utility-first responsive UI styling
Node.js	20.x LTS	Server-side JavaScript runtime
Express.js	4.x	REST API framework and middleware
MongoDB Atlas	7.x	Cloud-hosted document database
Mongoose	8.x	ODM for schema definition and queries
Socket.io	4.x	Real-time bidirectional event communication
jsonwebtoken	9.x	JWT generation and verification
bcryptjs	2.x	Password hashing and comparison
Axios	1.x	HTTP client for frontend API calls
Cloudinary SDK	Latest	Image upload, transformation, and CDN delivery
dotenv	16.x	Environment variable management
cors	2.x	Cross-Origin Resource Sharing configuration
Postman	11.x	API endpoint testing and documentation
Visual Studio Code	Latest	Primary development IDE
GitHub	—	Version control and collaboration

— End of Report —